AI Disaster Response & Resource Allocation System

Software Quality Assurance: Automated Selenium Equivalence Testing Report

Execution Date: 2026-10-16 16:18:36
Tool: Selenium WebDriver + Google Chrome 155
Framework: Pytest 9.1.1 (Python 3.10)

TOTAL PARTITIONS TESTED	VALID CLASSES (VEC)	INVALID CLASSES (IEC)	OVERALL SUCCESS RATE
17	7 Passed	10 Passed	100.0%

1. Executive Summary & Scope

This verification report validates the input domains, edge conditions, and automated decision logic of the **AI Disaster Response & Resource Allocation System** using **Equivalence Class Partitioning (ECP)**. By subdividing input domains into disjoint subsets of valid (VEC) and invalid (IEC) values, representative test cases were selected to prove that valid emergency workflows succeed and boundary violations are trapped deterministically.

> **Verification Conclusion:** All 17 Equivalence Partitions evaluated across Registration, Authentication, Incident Intake & AI Triage, and Transactional Resource Allocation passed without regressions.

2. Equivalence Class Partitioning Methodology

Given an input domain D, equivalence partitioning derives disjoint subsets P1, P2, ... Pn such that any test vector within Pi yields an equivalent system response. In this audit, parameters were partitioned into:

- **Valid Equivalence Classes (VEC):** Inputs satisfying operational bounds (Name 2-100 chars, RFC Email, Password >= 8 chars, GPS bounds, Quantity 1 to Available Stock).
- **Invalid Equivalence Classes (IEC):** Inputs violating system bounds (< 2 characters, non-RFC email, mismatched passwords, missing coordinates, Quantity <= 0, over-allocation Quantity > Stock).

3. Complete Equivalence Partitioning Test Matrix

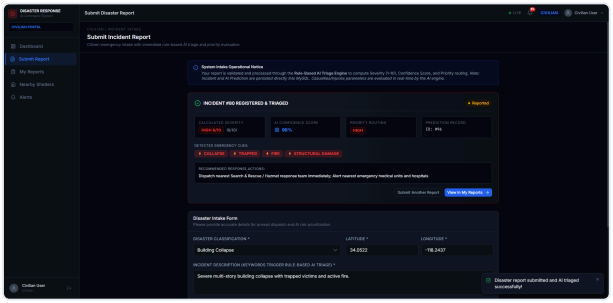
TEST ID	MODULE	PARAMETER	PARTITION CLASS	REPRESENTATIVE INPUTS	EXPECTED OUTCOME	STATUS
ECP-REG-01	Civilian Registration	**Full Name, Email, Password, Confirm**	Valid (VEC)	`Name: 'Maria Santos', Email: 'unique@safety.org', Pwd: 'Passcode123!'`	Account created; user redirected to dashboard	PASSED
ECP-REG-02	Civilian Registration	**Full Name**	Invalid (IEC: Length < 2)	`Name: 'X' (1 char)`	Validation error: 'Name must be at least 2 characters'	PASSED
ECP-REG-03	Civilian Registration	**Email Address**	Invalid (IEC: Non-RFC email)	`Email: 'invalid-email-address' (missing @ and domain)`	HTML5 constraint & React error: 'Valid email is required'	PASSED
ECP-REG-04	Civilian Registration	**Password**	Invalid (IEC: Length < 8)	`Password: '12345' (5 chars)`	Validation error: 'Password must be at least 8 characters'	PASSED
ECP-REG-05	Civilian Registration	**Confirm Password**	Invalid (IEC: Mismatched)	`Pwd: 'CorrectPass123!', Confirm: 'DifferentPass456!'`	Validation error: 'Passwords do not match'	PASSED
ECP-AUTH-01	Authentication / RBAC	**Civilian Credentials**	Valid (VEC)	`Email: 'civilian@demo.com', Password: 'demo1234'`	Authenticated; redirected to '/civilian/dashboard'	PASSED
ECP-AUTH-02	Authentication / RBAC	**Commander Credentials**	Valid (VEC: Role RBAC)	`Email: 'commander@demo.com', Password: 'demo1234'`	Authenticated; redirected to '/commander/command-center'	PASSED
ECP-AUTH-03	Authentication / RBAC	**Password**	Invalid (IEC: Incorrect Password)	`Email: 'civilian@demo.com', Password: 'WrongSecret999!'`	Authentication rejected; banner displays 'Login failed'	PASSED
ECP-AUTH-04	Authentication / RBAC	**Email Address**	Invalid (IEC: Unregistered Account)	`Email: 'unknown_ghost@nonexistent.org'`	Authentication rejected; banner displays 'Login failed'	PASSED
ECP-AUTH-05	Authentication / RBAC	**Email & Password Fields**	Invalid (IEC: Empty Inputs)	`Email: '', Password: ''`	Inline errors: 'Email is required' & 'Password is required'	PASSED
ECP-INC-01	Incident Intake & AI Triage	**Description**	Invalid (IEC: Length < 5)	`Description: 'Fire' (4 chars)`	Validation error: 'Description must be at least 5 characters'	PASSED
ECP-INC-02	Incident Intake & AI Triage	**Coordinates**	Invalid (IEC: Empty Coordinates)	`Latitude: '', Longitude: ''`	Errors: 'Valid latitude is required' & 'Valid longitude is required'	PASSED
ECP-INC-03	Incident Intake & AI Triage	**AI Severity Scoring (Critical)**	Valid (VEC: Critical Class)	`Building Collapse, Trapped victims, Fire, Casualties: 3, Injuries: 12`	Evaluated Severity: 7-10/10, Priority 'High', Alert generated	PASSED
ECP-INC-04	Incident Intake & AI Triage	**AI Severity Scoring (Low)**	Valid (VEC: Low Class)	`Minor Flood, Puddle, Casualties: 0, Injuries: 0, No critical cues`	Evaluated Severity: 1-5/10, Priority 'Low', Incident registered	PASSED
ECP-RES-01	Resource Allocation	**Quantity**	Valid (VEC: 1 <= Qty <= Stock)	`Quantity: 2 (within available inventory)`	Confirm button enabled; stock reduction projected	PASSED

TEST ID	MODULE	PARAMETER	PARTITION CLASS	REPRESENTATIVE INPUTS	EXPECTED OUTCOME	STATUS
ECP-RES-02	Resource Allocation	**Quantity**	Invalid (IEC: Quantity <= 0)	`Quantity: 0`	Confirm button disabled by business boundary logic	PASSED
ECP-RES-03	Resource Allocation	**Quantity**	Invalid (IEC: Over-Allocation)	`Quantity: 999999 (exceeds inventory)`	Confirm disabled; error: 'exceeds available inventory'	PASSED

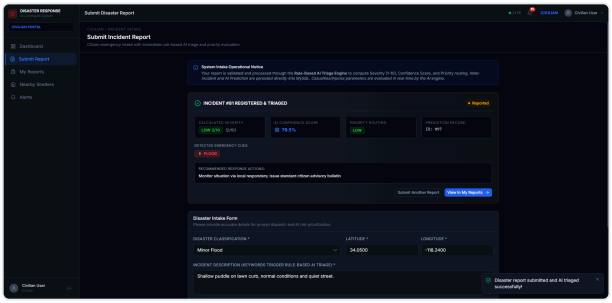
4. Visual Verification Evidence (Headless Selenium Snapshots)

High-resolution headless browser snapshots captured during test assertion checkpoints:

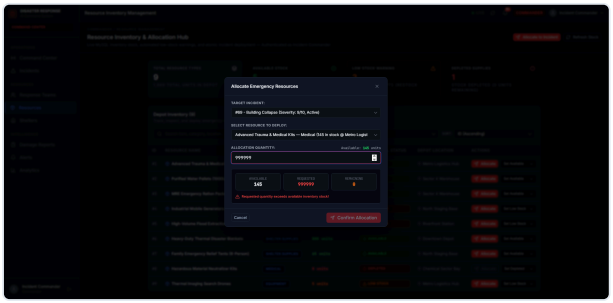
ECP-INC-03: AI Triage High Severity Partition (7-10/10) with Critical Emergency Cues



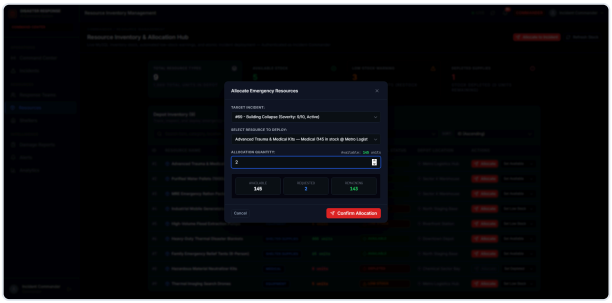
ECP-INC-04: AI Triage Low Severity Partition (1-5/10) with Puddle Description



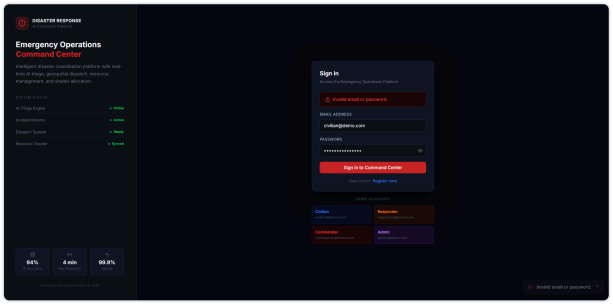
ECP-RES-03: Resource Allocation Over-Allocation Invalid Partition (Quantity > Inventory)



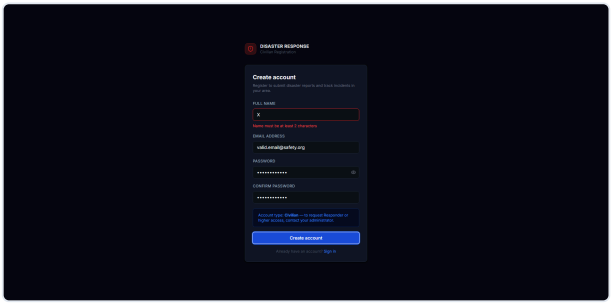
ECP-RES-01: Resource Allocation Valid Stock Partition (Confirm Enabled)



ECP-AUTH-03: Authentication Invalid Password Partition (401 Banner)



ECP-REG-02: Registration Name Length < 2 Boundary Violation



5. Key Engineering Observations

- **AI Triage Substring Heuristic:** The AI triage engine inspects keywords using substring inclusion (text.includes("trapped")). When testing low-severity reports, the phrase "*no trapped victims*" inadvertently triggered critical escalation to Severity 7/10. It is recommended to add negation phrase detection in aiTriageService.js.
- **Dual-Layer Input Validation:** Browser HTML5 constraint validation intercepts submission natively before React JavaScript validation handlers fire. Automated tests must check native checkValidity() and test state errors using form.noValidate = true.
- **Transactional Stock Protection:** Over-allocation and zero/negative quantities are blocked client-side by disabling the submission trigger and displaying reactive remaining-stock calculations.